Meeting Something New

System and AI Essentials | Lecture 13 | Part 1 of 2

Name: _____ Roll No: _____ Campus: _____

Let me tell you about a man you've never heard of

Fabrice Bellard is French, born in 1972, and he went to the École Polytechnique. At seventeen he wrote a program that compressed executable files. That's the least impressive thing on this page.

In December 2000 he started a project called FFmpeg, publishing his early work under a made-up name. It was a tool for handling video and audio, and at the time Linux was bad at both.

He also wrote QEMU, which is the machine emulator that sits underneath most cloud computing in the world. He wrote the Tiny C Compiler, small enough to compile itself in under a second. In 2009 he computed pi to 2.7 trillion digits, further than anyone had before, on an ordinary desktop PC rather than a supercomputer.

John Carmack, who wrote Doom, has said publicly that Bellard is almost certainly a better programmer than he is.

Here's the part that matters to you.

FFmpeg is inside VLC. It's inside your browser. It's in the pipeline that processes video for basically every platform you use. Every single video you have ever watched has passed through code this man wrote.

And until ninety seconds ago, you had never heard his name.

So here's the problem

Nobody is going to teach you the tools that matter.

There are hundreds of thousands of them. New ones appear every week. Your instructors don't know most of them, and the ones we do know will be different by the time you graduate. A course that taught you tools would be out of date before you finished it.

So we're going to do something else today.

Today you're not building anything. You're going to meet two tools you've never used, learn them in the room, and leave with a method for doing it again on your own. That method is the point. The tools are just what we're practising on.

> You will produce things today. A GIF from a video you shot. A QR code that goes to your own work. Those are proof the method worked, not the goal.

Before I show you anything

Here's the situation you'll be in for the rest of your career.

Somebody hands you a tool you've never heard of and says the answer is in there somewhere. No tutorial. No video. No one to ask. There's documentation, and there's your terminal, and that's it.

What do you do first? And after that? Write your steps. Four to six lines, and guess if you have to.

1. _____

2. _____

3. _____

4. _____

5. _____

6. _____

Don't turn the page until you've written at least four. You did this in Lecture 12 and it worked. Same deal.

The five steps

This is the whole method. It fits on one line each.

Step	Name	What you're actually doing
1	What is this for	Read the docs. One sentence, your words.
2	Find the shape	What does a command look like? Copy the pattern.
3	Run the smallest thing that works	Not the impressive thing. The smallest.
4	Change exactly one thing	Predict, change, run, explain the gap.
5	Break it on purpose	Find out what it says when you're wrong.

Which of these did you not have?

> Step 5 is the one almost nobody writes down. Breaking something deliberately, while you're calm and nothing's at stake, is how you learn to read its errors later when something's on fire.

We're going to run these five steps twice today. Once together, slowly, on a big tool. Then again, faster, on a small one. If the same five steps work on two completely different tools, they'll work on the next one too, and that's the thing you're taking home.

How I'm going to help you today

Same as last time. I'll tell you what you're looking for and where to look, and you'll find it. Every hunt looks like this:

Go find out

Find out. The question.
Where I'd look. A docs page, a help flag, a manual. Never the answer.
Shape of a right answer. If what you've got doesn't match this, you're off track.
How you'll know. What you should be seeing.

Sometimes you'll get the word length too, like this: □□□□□□ □□□. One square per letter.

Tool one: ffmpeg

First, get something to work on

You need a short video. Three seconds is plenty. Ten is too many.

Any machine

Record your screen for a few seconds. On macOS that's Shift-Command-5. On Linux, use whatever your desktop provides.

Or use the clip your instructor is handing out. Nobody is being marked on cinematography.

Move it into a folder you made, in the terminal.

Filename: _____ File size: _____

Install it

macOS

`brew install ffmpeg`. If `brew` isn't there, put your hand up now rather than in ten minutes.

Linux and WSL

```
sudo apt install ffmpeg
```

This is a big download. While it runs, read the front page of `ffmpeg.org`. That's Step 1 and you're getting it for free.

Step 1. What is this for

Find out. What does ffmpeg do? Not what you're about to use it for. What is it *for*.

Where I'd look. The front page of `ffmpeg.org`, and the first paragraph of `man ffmpeg`.

Shape of a right answer. One sentence. Your words, not theirs. If you've copied a sentence with the word "multimedia framework" in it, you've skipped the thinking.

How you'll know. You could say it out loud to someone who doesn't code.

In your own words:

Hands up. Three answers, out loud. They will all be different, and that's the point.

Step 2. Find the shape

Find out. What does an ffmpeg command look like? What are its parts, and in what order?

Where I'd look. `ffmpeg -h`. It's long. You want the very top, the line that shows the pattern. Stop reading after that.

Shape of a right answer. A pattern with placeholders in it, not a real command.

How you'll know. You can point at which part is the input and which part is the output.

Copy the pattern exactly as it appears:

Which part is the input? _____

Which part is the output? _____

Notice what decides the output format. It isn't a flag. It's the file extension you type. That's unusual and it's worth knowing.

Step 3. Run the smallest thing that works

Find out. How do you pull a single frame out of your video and save it as an image?

Where I'd look. The docs, or `ffmpeg -h`, for a flag that limits how many video frames get written.

Shape of a right answer. Your input file, one flag with the number 1, and an output filename ending in `.jpg`.

How you'll know. A picture appears in your folder. Open it.

Your command:

Did it work? **Yes** / **No** Output file size: _____

That's the whole game, right there. You just used a tool you'd never opened, from documentation, in about four minutes. Everything after this is variations.

Step 4. Change exactly one thing (three times)

Now we go from a still image to a GIF. Three changes, one at a time. **Predict before each run.**

Change 1: make it a GIF. Change nothing but the output extension, and drop the frame limit.

Predict what will happen: _____

Your command:

What actually happened, and how big is the file? _____

If your GIF is enormous, that's correct and it's the problem we're about to fix. Don't open it in a browser yet.

Change 2: make it shorter. Find the flag that limits duration to a few seconds.

Where I'd look. `ffmpeg -h`, in the main options. It's a single letter and it takes a number of seconds.

Your command:

New file size: _____ How much smaller? _____

Change 3: make it smaller and smoother. This one uses a video filter, which is a different kind of option and worth meeting now.

> The flag is `-vf` and it takes a string with the filters inside it, separated by commas. Two filters you want are one that sets frames per second and one that scales. Search the ffmpeg filters documentation for `fps` and `scale`. For scale, a width with `-1` for the height keeps the proportions.

Your command:

Final file size: _____

Version	File size	What changed
Single frame (jpg)		
First GIF		
Shortened		
Scaled and fps-limited		

Open the final GIF. Does it look right?

> **Everyone up.** Walk around, look at three other people's GIFs. Two minutes.

Step 5. Break it on purpose

Your instructor will call one. Run it, and write down the **first line of the error** plus what it's telling you.

Break 1: ask for a format it can't produce. Change your output extension to something ffmpeg can't write, like `.xyz`.

First line:

What it's telling you:

Break 2: misspell a filter name inside your `-vf` string.

First line:

What it's telling you:

---

ffmpeg prints a wall of text before it gets to the error. That's not it hiding the problem. The wall is a report on your file, which is useful ninety percent of the time. Learn to scroll to the bottom.

Tool two: qrencode

Different tool. Different job. Same five steps, and this time you're on your own for most of it.

Install

macOS: `brew install qrencode` Linux and WSL: `sudo apt install qrencode`

This one is small. It'll be done before you've finished reading this box.

Step 1. What is this for? One sentence, your words.

Step 2. Find the shape. Where would you look? _____

The pattern:

Step 3. Smallest thing that works. Make a QR code out of any short piece of text.

You need to tell it where to put the output. That's a flag, and it's the same letter most tools use for that. □□

Your command:

Step 4. Change one thing. Now make a QR code of **your own GitHub repository URL** from Lecture 12.

Your command:

If the text has slashes or colons in it, wrap it in quotes. The shell will otherwise try to interpret some of those characters itself.

Step 5. Break it on purpose. Your choice this time. What did you try, and what did it say?

Everyone up again. Scan somebody else's QR code with your phone. You should land on their repository from Lecture 12.

Two tools. Neither one taught to you. Both working, in one session, from documentation. That's not a small thing and you should notice you just did it.

The only thing you're committing today

You didn't write a program today, so there's no program to push.

What you're pushing is what you learnt.

Start a learning log

Make a new repository. Call it `learning-log`, or whatever you like. This one isn't for a project. It's a place you write down every tool you meet, for as long as you're here.

Three years from now this repository will be the most useful thing in your GitHub account, and not because anyone else reads it. In six months you'll need ffmpeg again, you'll remember you did this once, and past-you will have left instructions.

Write the README

Six things. Keep it short. This is a note to yourself, not an essay.

1. **The date, and the two tools you met.**
2. **What each one is for.** One sentence each, your words.
3. **The commands that worked.** Copy them out of this sheet. Real commands, with real filenames, the way they actually ran.
4. **What broke, and what fixed it.** At least two.
5. **The five steps.** Written out, because next time this page won't be in front of you.
6. **One thing you'd want to try next.**

Point 3 is the one that matters. "I used ffmpeg to make a GIF" is useless to you in June. The actual command, with the actual flags, is worth ten minutes of your life every time you come back to it.

Commit and push

Commit message: _____

Short hash: _____ Repository URL: _____

Are you committing the GIF and the QR image too? ☐ Yes ☐ No

Why:

> Worth thinking about before you decide. How big is that GIF? A repository is not a good place to keep large files, and it never forgets them once they're in.

Before you go

Go back to page one and read the steps you wrote before you knew anything.

What would you add?

What would you throw out?

Which of the five steps was hardest, and why?

> Tomorrow you do it alone, in Python, with AI helping. Same five steps. No demonstration.

If you're done early

1. Make a GIF that plays forwards and then backwards. The filter you want is in the ffmpeg filters documentation and it is not obvious.
2. Extract every frame of your clip as a separate image. Then count them in the terminal.
3. Make a QR code big enough to read from across the room, then work out which flag controls that.
4. Find one more flag in `ffmpeg -h` that you can explain to someone. Add it to your README.

Where the story comes from

- Fabrice Bellard's own site, `bellard.org`, which lists his projects including FFmpeg, QEMU, TinyCC and the pi computation.
- The FFmpeg project, `ffmpeg.org`. First release 20 December 2000.
- The Computer History Museum's record of the pi computation announcement, January 2010.

Instructor section

Check	Value
Final GIF file size recorded	
Four sizes in the comparison table, all different	
Learning log repo URL present and reachable	
README contains real commands, not descriptions	

Defense probe, pick one:

- Point at one flag in their final ffmpeg command. What does it do?
- Your GIF got smaller three times. Which change made the biggest difference, and why?
- Read me point 3 of your README. Could you run that command in six months from what you wrote?

Answered adequately: **Yes / No** Instructor initials: _____